Reporte

Nombre:	**Matrícula**:
Valeria Ordaz Garcia Luis José Mendoza Serrano Omar Alejandro Salazar Castro	2169894 2223173 2226576
Nombre del curso: Componentes de lógica del negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: Diseño Arquitectural - Bitacora ia
Fecha: 12/10/2026	

Bibliografia (Formato APA)

Documentos impartidos por el profesor

- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gotel, O. C. Z., & Finkelstein, A. C. W. (1994). An analysis of the requirements traceability problem. En *Proceedings of the First International Conference on Requirements Engineering* (pp. 94-101). IEEE.
- Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect.
- Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
- Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Herramientas de IA

- Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai
- OpenAI. (2026). *ChatGPT* [Modelo de lenguaje de gran tamaño]. https://chat.openai.com
- DeepSeek. (2026). *DeepSeek* [Modelo de lenguaje de gran tamaño]. https://www.deepseek.com

Objetivo:

Documentar y justificar el uso de herramientas de inteligencia artificial (Claude y ChatGPT) durante el diseño arquitectural del sistema de EduTechAcademy, registrando en cada iteración el objetivo, el prompt empleado, la respuesta útil obtenida, el error o supuesto detectado y la decisión tomada por el equipo. Con esto se evidencia que la IA funcionó como apoyo supervisado, que cada propuesta se contrastó con el BRD, las reglas de negocio y los ADR, y que las correcciones y decisiones finales fueron responsabilidad del equipo.

Procedimiento:

1. Resumen de la bitácora

Proyecto: Diseño Arquitectural del Sistema
Empresa: EduTechAcademy
Herramienta de IA utilizada: Claude (Anthropic) / ChatGPT/ DeepSeek

2. Bitácora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	**Etapa 1:** Definir los casos de uso y sus flujos principales, alternativos y de rechazo.	"Con el BRD de EduTechAcademy, las User Stories (US-01 a US-18) y sus criterios de aceptación, identifica los casos de uso que cubren el ciclo del estudiante: inscripción, impartición, evaluación y certificación. Para cada uno indica actor, objetivo, entradas, reglas, resultado y flujos alternativos, vinculándolos con BN, US y CA."	Se consolidaron los CU-01 a CU-05 con escenarios normales y excepciones. Se identificaron los actores principales: Estudiante, Instructor, Control Escolar, Administración y Cliente empresarial.	La IA asumió que todos los cursos tenían los mismos requisitos de inscripción. No consideró que los requisitos pueden variar por curso (RB-01).	Se ajustó el CU-02 para incluir la validación de requisitos específicos por curso mediante `PoliticalInscripcion.verificar()`.
2	**Etapa 2:** Comparar patrones de lógica de negocio y justificar la selección.	"Compara Transaction Script, Table Module y Domain Model para EduTechAcademy. Considera la complejidad de las reglas de certificación (70/100 y 80%), la necesidad de trazabilidad, los estados que dependen unos de otros y las reglas que cruzan inscripción, grupo y pago. Presenta ventajas, riesgos y evidencia del BRD."	La comparación condujo a seleccionar Domain Model como patrón principal. Se elaboró una tabla comparativa con 10 criterios y puntuaciones (Transaction Script: 11/30, Table Module: 17/30, Domain Model: 28/30).	La IA no consideró el costo inicial de aprendizaje ni la curva de adopción del equipo.	Se aceptó Domain Model, pero se documentó el costo en el ADR-001 y se decidió no usarlo para consultas ni reportes (ADR-008).
3	**Etapa 3:** Representar visualmente los conceptos principales del negocio y sus relaciones (Mapa de Dominio).	"Convierte la arquitectura de dominio validada de EduTechAcademy en un mapa de dominio editable en Draw.io. Usa como base el lenguaje ubicuo, las entidades, los Value Objects, los siete Aggregates con sus raíces, los Domain Services y los Domain Events definidos en la documentación. No agregues clases, reglas ni relaciones que no estén respaldadas por el modelo. Organiza el diagrama para que sea legible y distingue los elementos por tipo."	Se generó un mapa de dominio con los 7 Aggregates (Curso, Grupo, Instructor, Estudiante, Inscripción, SolicitudCapacitación, ClienteEmpresarial), sus relaciones por identificador y los Domain Events principales.	La IA sugirió que Pago fuera un Aggregate separado de `Inscripción`. Esto rompía la lógica de certificación, que necesita el pago junto con la inscripción.	Se rechazó la separación. Pago se mantuvo como entidad dentro del Aggregate `Inscripción` para que `evaluarCertificacion()` tenga todos los datos juntos (ADR-002).

Reporte

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
3.1	**Etapa 3:** Corregir el mapa de dominio, el lenguaje ubicuo, los agregados y los servicios de dominio.	"Revisa el mapa contra las reglas RB-06, RB-07, RB-08 y los criterios de certificación. La elegibilidad de un estudiante no debe perderse por una pausa en el pago. Evalúa separar los motivos de cancelación/cierre, conserva las decisiones que tengan sustento y marca lo no definido como pendiente. No afirmes haber revisado el BPMN porque no está disponible."	Se revisaron las 7 raíces y los 2 Domain Services. Se identificó el riesgo de mezclar la cancelación del curso con el cierre de una inscripción. Se documentaron los supuestos SUP-09 a SUP-15 como pendientes.	La IA tendía a inventar reglas no documentadas para completar el modelo (por ejemplo, plazos de pago no definidos).	Se documentaron los supuestos SUP-09 a SUP-15 como pendientes de confirmar con el docente o stakeholder. No se presentaron como reglas aprobadas.
4	**Etapa 4:** Revisar la Service Layer y la coordinación mediante Domain Events.	"Audita los Application Services para que coordinen los casos de uso sin acumular reglas de dominio. Revisa los métodos de inscripción, pago, certificación y cambio de grupo. Evita notificaciones duplicadas y responde las preguntas de revisión de la guía con evidencia."	Se identificaron métodos que modificaban más de un Aggregate en una sola transacción sin justificación, una notificación duplicada en `PagoRecibido` y responsabilidades repetidas en `InscripcionService`.	La IA puso la validación de cupo dentro del servicio `InscripcionService.inscribir()`. Esto violaba el principio de que el dominio debe decidir.	Se corrigió: el servicio solo llama a `Grupo.ocuparLugar()`. La validación de cupo está en el Value Object Cupo dentro del Aggregate Grupo (ADR-002).
4.1	**Etapa 4:** Corregir el proceso de cancelación por saldo vencido.	"Conforme a RB-10, si un curso se cancela, los estudiantes deben ser notificados y pueden solicitar reembolso o transferencia. El Coordinador (Control Escolar) registra la cancelación con su motivo. Distingue este proceso del cierre por falta de pago y actualiza los componentes afectados."	Se separó el aviso de cancelación de la acción de registrar la cancelación. Se detectó un defecto en el orden de verificación del pago en `solicitarReembolso()`.	La IA proponía una cancelación automática por falta de pago. Esto contradecía la regla de negocio: Control Escolar debe autorizar y registrar la cancelación.	Se corrigió: `Grupo.cancelar()` se confirma en una transacción y publica `GrupoCancelado`. El manejador de eventos actualiza las inscripciones y notifica a los estudiantes (ADR-007).
5	**Etapa 5:** Definir Repository, Unit of Work y límites transaccionales.	"Define interfaces Repository por raíz de agregado, separadas de la infraestructura. Completa la tabla de Unit of Work con inicio, cambios confirmados juntos y manejo de errores; incluye el cierre por cancelación de curso y explica cómo reducir el riesgo de perder Domain Events tras el commit."	Se organizaron las interfaces por Aggregate: `CursoRepository`, `GrupoRepository`, `InscripcionRepository`, `EstudianteRepository`, `SolicitudRepository`, `ClienteRepository`, `InstructorRepository`. Se propuso un Outbox para manejar eventos después del commit.	La IA sugirió usar una sola transacción para todas las operaciones. Esto violaba el límite de un Aggregate por transacción.	Se definió que cada operación modifica una raíz por transacción, con excepciones documentadas (inscribir, cambiarGrupo) que se ejecutan en una sola transacción por necesidad de invariantes (ADR-006).
6	**Etapa 6:** Aplicar CQRS básico	"Aplica CQRS sin bases de datos separadas. Separa	Se separaron Commands, Queries y	La IA propuso un `SolicitudPendiente`	Se ajustó el View Model para no duplicar esa información.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
	separando comandos, consultas y modelos de lectura.	Commands y Queries, evita duplicar datos financieros en los View Models y haz que `ConsultarEstadoPagosQuer` y reciba `eventold` y consulte las cuentas correspondientes."	Handlers. Se ajustaron los View Models para evitar duplicar información financiera. `ConsultarEstadoPagosHandler` recibe `eventold` y consulta las cuentas correspondientes.	`ViewModel` que incluía datos financieros que correspondían al estado de pagos.	Los datos financieros se consultan por separado desde `EstadoCuenta` (ADR-008).
7	**Etapa 7:** Evaluar SOLID y construir la matriz de trazabilidad y el código base.	"Revisa las etapas 3 a 6 con los cinco principios SOLID. Registra hallazgo, evidencia, impacto y corrección o justificación. Construye la matriz BN, SE, US, CA y CU sin inventar relaciones y organiza el código en domain, application e infrastructure."	Se documentaron hallazgos H-01 a H-10. Se preparó una base Python en tres capas. Se identificaron relaciones de trazabilidad inferidas porque no se disponía de BPMN.	Algunas relaciones de trazabilidad quedaron inferidas porque no se disponía de BPMN. La IA intentó completar relaciones no documentadas.	Se dejaron las relaciones inferidas marcadas como pendientes. No se inventaron reglas para acomodar la arquitectura.
8	**Etapa 8:** Representar la arquitectura con UML y ADR.	"Genera un borrador de diagrama de clases en PlantUML para el dominio de EduTechAcademy. Incluye los 7 Aggregates, sus entidades internas, Value Objects, el Domain Service y los Application Services. No agregues clases, reglas o relaciones que no aparezcan en la descripción."	Se generó un borrador de diagrama de clases con los elementos principales. Se identificaron las relaciones de composición y las referencias por identificador.	La IA incluyó una clase Pago como Aggregate separado. También incluyó métodos que no estaban en el diseño.	Se corrigió el diagrama: Pago es una entidad dentro de `Inscripción`. Se eliminaron métodos no documentados. Se usó PlantUML para la versión final.
9	**Etapa 9:** Validar la propuesta con pruebas automatizadas.	"Convierte los casos de uso CU-01 a CU-05 en pruebas unitarias con unittest para Python. Prueba flujos principales, alternativos y de error. Verifica que las reglas de negocio se cumplan y que los límites transaccionales se respeten."	Se generaron 5 módulos de pruebas: `test_coherencia_diagramas.py` (14 pruebas), `test_correcciones_etapas_3_a_7.py` (12 pruebas), `test_etapa9_casos_de_uso.py` (24 pruebas), `test_reglas_corregidas.py` (10 pruebas). Total: 70 pruebas.	La IA generó pruebas que fallaban porque buscaban archivos `.docx` en carpetas incorrectas. También hubo errores en nombres e importaciones.	Se corrigieron los errores. Se ajustó la búsqueda para excluir la carpeta `tests/evidencias/`. Se ejecutó la suite completa: 70 pruebas aprobadas, 0 fallidas, 0 errores, 0 omitidas (0.620 s).
10	Revisión de la documentación y consistencia entre diagramas y código.	"Revisa que los métodos, clases, estados y Domain Events del mapa de dominio existan en el código. Que los documentos citen métodos existentes y usen una sola versión del modelo. Que la matriz de trazabilidad no tenga caracteres invisibles ni	Se encontraron inconsistencias entre el mapa de dominio y el código. Algunos métodos no existían. La matriz de trazabilidad tenía identificadores mal formados.	La IA no detectó todas las inconsistencias. Algunos nombres de métodos diferían entre documentos.	Se corrigieron los nombres de métodos y se unificó el lenguaje ubicuo. Se actualizó la matriz de trazabilidad.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
		identificadores mal formados."			
11	Revisión de los ADR y su estado.	"Revisa los 11 ADR. Distingue una decisión aceptada de una propuesta pendiente. Asegúrate de que cada ADR tenga contexto, alternativas, decisión, consecuencias y trazabilidad."	Se identificó que algunos ADR no tenían trazabilidad explícita con las reglas del BRD. Se ajustaron los estados de los ADR.	La IA tendía a marcar todos los ADR como "Aceptada". Algunos eran propuestas pendientes de aprobación del equipo.	Se diferenciaron los estados: ADR-001 a ADR-011 con estado "Aceptada" (excepto ADR-002 y ADR-004, que quedaron como "Propuesta" en el equipo de Eventos Nova). En EduTechAcademy, todos quedaron como "Aceptada".
12	Revisión final de la bitácora de IA y preparación de la entrega.	"Organiza la bitácora de IA con los 6 campos solicitados: iteración, objetivo, prompt, respuesta útil, error o supuesto, decisión del equipo. Asegúrate de que cada decisión esté respaldada por evidencia del BRD y los documentos de la Unidad 2."	Se consolidó la bitácora con 16 registros (12 principales + 4 revisiones intermedias). Se documentaron los errores detectados y las correcciones realizadas.	La IA no conservó todos los detalles de las revisiones intermedias. Algunos prompts originales no estaban documentados.	Se reconstruyeron los prompts a partir de los documentos y se documentaron las decisiones humanas. Se reconoce que el historial original no conserva todos los detalles.

3. Revisiones intermedias (adicionales)

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
R1	Revisión de la comparación de patrones.	"Revisa la tabla comparativa de patrones. ¿Falta algún criterio relevante para EduTechAcademy? ¿Las puntuaciones son justas?"	Se agregó el criterio "Encaje con la Unidad 2 (Aggregates, Value Objects, Domain Events, Service Layer)". Se ajustaron las puntuaciones de Transaction Script (11/30) y Table Module (17/30).	La IA no había considerado el criterio de encaje con la Unidad 2.	Se agregó el criterio 10 y se recalculó la puntuación total.
R2	Revisión de la definición de umbrales (70/100 y 80%).	"Revisa que los umbrales de certificación estén en un solo lugar. ¿Deberían ser Value Objects o constantes?"	Se confirmó que los umbrales deben ser Value Objects (`Calificación` y `PorcentajeAsistencia`) para evitar duplicación.	La IA sugirió usar constantes en el Service Layer. Esto violaba el ADR-003.	Se mantuvieron como Value Objects dentro del Aggregate `Inscripción`.
R3	Revisión de los límites transaccionales.	"Revisa los límites transaccionales. ¿Hay operaciones que modifiquen más de un Aggregate sin justificación?"	Se identificó que `registrarAsistencia` y `validarCertificacion` modifican varias inscripciones en una sola transacción. Es un trade-off aceptado porque el número está acotado por el cupo.	La IA sugirió usar consistencia eventual para el cupo. Esto permitiría sobreventa y violaría RB-02.	Se mantuvo la transacción única para el cupo. Se documentó el trade-off en ADR-006.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
R4	Revisión de la matriz de trazabilidad.	"Revisa que la matriz de trazabilidad relacione correctamente BN, US, CU, RB y ADR. ¿Falta alguna relación?"	Se agregaron las relaciones entre REP-01 a REP-10 y los Application Services. Se vincularon los ADR con los problemas del BRD (PA-01 a PA-04).	La IA no había incluido los reportes en la matriz.	Se agregó la sección de trazabilidad de reportes y se actualizó la matriz.

4. Resumen de errores detectados y corregidos

Error detectado	Iteración	Corrección aplicada	ADR relacionado
Separar Pago como Aggregate independiente.	3	Se mantuvo Pago como entidad dentro de `Inscripción`.	ADR-002
Validar cupo dentro del Service Layer.	4	Se movió la validación al Value Object Cupo dentro de `Grupo`.	ADR-002
Cancelación automática por falta de pago.	4.1	Control Escolar registra la cancelación con su motivo.	ADR-007
Usar una sola transacción para todas las operaciones.	5	Cada operación modifica una raíz por transacción (con excepciones documentadas).	ADR-006
Duplicar datos financieros en View Models.	6	Se ajustaron los View Models para no duplicar información.	ADR-008
Usar constantes en lugar de Value Objects para umbrales.	R2	Se mantuvieron como Value Objects (`Calificación`, `PorcentajeAsistencia`).	ADR-003
Incluir clases y métodos no documentados en el diagrama.	8	Se corrigió el diagrama con PlantUML.	ADR-001, ADR-002
Pruebas que fallaban por buscar archivos en carpetas incorrectas.	9	Se ajustó la búsqueda para excluir `tests/evidencias/`.	-
Inconsistencias entre mapa de dominio y código.	10	Se unificaron nombres y se actualizó el lenguaje ubicuo.	ADR-001
ADR sin trazabilidad explícita.	11	Se agregó trazabilidad con BN, US, CA y CU.	ADR-001 a ADR-011